

## Week – 8 skill

### Session - 15

#### TASK 1 — Combined Redirection, File Descriptors & Merging Streams

Create a sample file

```
ganesh@DESKTOP-79N3772:~$  
ganesh@DESKTOP-79N3772:~$ printf "Apple\nBanana\nMango\n" > fruits.txt  
ganesh@DESKTOP-79N3772:~$ cat fruits.txt  
Apple  
Banana  
Mango  
ganesh@DESKTOP-79N3772:~$
```

Redirect standard output

```
Apple  
Banana  
Mango  
ganesh@DESKTOP-79N3772:~$ echo "Hello Ubuntu" > output.txt  
ganesh@DESKTOP-79N3772:~$ cat output.txt  
Hello Ubuntu  
ganesh@DESKTOP-79N3772:~$
```

Append output

```
ganesh@DESKTOP-79N3772:~$ echo "Second Line" >> output.txt  
ganesh@DESKTOP-79N3772:~$ cat output.txt  
Hello Ubuntu  
Second Line  
ganesh@DESKTOP-79N3772:~$
```

Redirect error

```
Hello Ubuntu  
Second Line  
ganesh@DESKTOP-79N3772:~$ ls existing.txt 2> error.txt  
ganesh@DESKTOP-79N3772:~$ cat error.txt  
ls: cannot access 'existing.txt': No such file or directory  
ganesh@DESKTOP-79N3772:~$
```

Merge standard output and error

```

ganesh@DESKTOP-79N3772:~$ ls existing.txt 2> error.txt
ganesh@DESKTOP-79N3772:~$ cat error.txt
ls: cannot access 'existing.txt': No such file or directory
ganesh@DESKTOP-79N3772:~$ ls fruits.txt missing.txt > result.txt 2>&1
ganesh@DESKTOP-79N3772:~$ cat result.txt
ls: cannot access 'missing.txt': No such file or directory
Fruits.txt
ganesh@DESKTOP-79N3772:~$

```

Combine output and error using &>

```

ganesh@DESKTOP-79N3772:~$ ls fruits.txt missing.txt &> combined.txt
ganesh@DESKTOP-79N3772:~$ cat combined.txt
ls: cannot access 'missing.txt': No such file or directory
fruits.txt
ganesh@DESKTOP-79N3772:~$

```

Duplicate file descriptor using tee

```

ganesh@DESKTOP-79N3772:~$ echo "System test successful" | tee test.txt
System test successful
ganesh@DESKTOP-79N3772:~$ cat test.txt
System test successful
ganesh@DESKTOP-79N3772:~$

```

Verify ordering

```

ganesh@DESKTOP-79N3772:~$ cat test.txt
System test successful
ganesh@DESKTOP-79N3772:~$ printf "3\n1\n2\n" > numbers.txt
ganesh@DESKTOP-79N3772:~$ sort numbers.txt
1
2
3
ganesh@DESKTOP-79N3772:~$ cat fruits.txt numbers.txt > merged.txt 2> errors.txt
ganesh@DESKTOP-79N3772:~$ cat merged.txt
Apple
Banana
Mango
3
1
2
ganesh@DESKTOP-79N3772:~$

```

## TASK 1 — Short Observation

**Observation:** Standard input/output/error redirection was performed using >, >>, 2>, 2>&1, and &>. Multiple outputs were merged and file-descriptor redirection was verified successfully.

## TASK 2 — Complex Commands, Pipes & Redirections

```

ganesh@DESKTOP-79N3772:~$ printf "Java\nPython\nJava\nC\nPython\nJava\n" > languages.txt
ganesh@DESKTOP-79N3772:~$ cat languages.txt
Java
Python
Java
C
Python
Java
ganesh@DESKTOP-79N3772:~$

```

Use a pipe

```

ganesh@DESKTOP-79N3772:~$ cat languages.txt | sort
C
Java
Java
Java
Python
Python
ganesh@DESKTOP-79N3772:~$ cat languages.txt | wc -l
6
ganesh@DESKTOP-79N3772:~$ cat languages.txt | grep "Java" | wc -l
3
ganesh@DESKTOP-79N3772:~$ cat languages.txt | sort | uniq
C
Java
Python
ganesh@DESKTOP-79N3772:~$ cat languages.txt | sort | uniq > unique.txt
ganesh@DESKTOP-79N3772:~$ cat unique.txt
C
Java
Python
ganesh@DESKTOP-79N3772:~$
ganesh@DESKTOP-79N3772:~$ cat languages.txt missing.txt 2> errors.txt | sort > sorted.txt
ganesh@DESKTOP-79N3772:~$ cat sorted.txt
C
Java
Java
Java
Python
Python
ganesh@DESKTOP-79N3772:~$ cat errors.txt
cat: missing.txt: No such file or directory
ganesh@DESKTOP-79N3772:~$
ganesh@DESKTOP-79N3772:~$ cat languages.txt | sort | uniq -c
1 C
3 Java
2 Python
ganesh@DESKTOP-79N3772:~$

```

## TASK 2 — Short Observation

**Observation:** Complex Linux commands were executed using pipes, redirection, `grep`, `sort`, `uniq`, and `wc`. Command execution order and separate error handling were successfully verified.

## TASK 1 — Detect Background Jobs & Launch Non-Blocking Processes

### Question

Write and execute Linux commands to detect background jobs, launch non-blocking processes, return the prompt immediately, store job information, monitor execution, and track background tasks.

```
ganesh@DESKTOP-79N3772:~$ sleep 30 &
[1] 968
ganesh@DESKTOP-79N3772:~$ jobs
[1]+  Running                  sleep 30 &
ganesh@DESKTOP-79N3772:~$
```

```
ganesh@DESKTOP-79N3772:~$ jobs
[1]+  Running                  sleep 30 &
ganesh@DESKTOP-79N3772:~$ jobs > jobinfo.txt
ganesh@DESKTOP-79N3772:~$ cat jobinfo.txt
[1]+  Done                      sleep 30
ganesh@DESKTOP-79N3772:~$ ps
  PID TTY          TIME CMD
   362 pts/0        00:00:00 bash
  1006 pts/0        00:00:00 ps
ganesh@DESKTOP-79N3772:~$ sleep 60 &
[1] 1016
ganesh@DESKTOP-79N3772:~$
```

```
ganesh@DESKTOP-79N3772:~$ sleep 30 &
[1] 968
ganesh@DESKTOP-79N3772:~$ jobs
[1]+  Running                  sleep 30 &
ganesh@DESKTOP-79N3772:~$ jobs > jobinfo.txt
ganesh@DESKTOP-79N3772:~$ cat jobinfo.txt
[1]+  Done                      sleep 30
ganesh@DESKTOP-79N3772:~$ ps
  PID TTY          TIME CMD
   362 pts/0        00:00:00 bash
  1006 pts/0        00:00:00 ps
ganesh@DESKTOP-79N3772:~$ sleep 60 &
[1] 1016
ganesh@DESKTOP-79N3772:~$ jobs
[1]+  Running                  sleep 60 &
ganesh@DESKTOP-79N3772:~$ ps -ef | grep sleep
ganesh    1038   362  0 03:53 pts/0    00:00:00 grep --color=auto sleep
[1]+  Done                      sleep 60
ganesh@DESKTOP-79N3772:~$ jobs
ganesh@DESKTOP-79N3772:~$ jobs
ganesh@DESKTOP-79N3772:~$
```

### TASK 1 — Final Observation

**Observation:** Background processes were launched using `&`, detected using `jobs`, monitored using `ps`, stored in a file, and terminated using `kill`.

## TASK 2 — Create Job Table, Store Process Groups, Track States & Update Status

### Question

Write and execute Linux commands to create a job/process table, store process-group information, track process states, update process status, remove completed jobs, and validate job records.

```
[1] 1068
ganesh@DESKTOP-79N3772:~$ echo "JOB_ID PID STATE COMMAND" > job_table.txt
ganesh@DESKTOP-79N3772:~$ ps -o pid=,stat=,comm= -C sleep | awk '{print "1 \"$1\" \"$2\" \"$3}' >> job_table.txt
ganesh@DESKTOP-79N3772:~$ cat job_table.txt
JOB_ID PID STATE COMMAND
1 1068 S sleep
ganesh@DESKTOP-79N3772:~$
```

```
ganesh@DESKTOP-79N3772:~$ echo "JOB_ID PID STATE COMMAND" > job_table.txt
ganesh@DESKTOP-79N3772:~$ ps -o pid=,stat=,comm= -C sleep | awk '{print "1 \"$1\" \"$2\" \"$3}' >> job_table.txt
ganesh@DESKTOP-79N3772:~$ cat job_table.txt
JOB_ID PID STATE COMMAND
1 1068 S sleep
ganesh@DESKTOP-79N3772:~$ ps -o pid,pgid,ppid,stat,cmd -C sleep
  PID   PGID   PPID STAT CMD
  1068   1068    362 S   sleep 100
ganesh@DESKTOP-79N3772:~$ ps -o pid,pgid,ppid,stat,cmd -C sleep
  PID   PGID   PPID STAT CMD
  1068   1068    362 S   sleep 100
[1]+  Done                  sleep 100
ganesh@DESKTOP-79N3772:~$
```

```
ganesh@DESKTOP-79N3772:~$ ps -o pid,pgid,ppid,stat,cmd -C sleep
  PID   PGID   PPID STAT CMD
  1068   1068    362 S   sleep 100
ganesh@DESKTOP-79N3772:~$ ps -o pid,pgid,ppid,stat,cmd -C sleep
  PID   PGID   PPID STAT CMD
  1068   1068    362 S   sleep 100
[1]+  Done                  sleep 100
ganesh@DESKTOP-79N3772:~$ ps -o pid,stat,cmd -C sleep
  PID STAT CMD
  1068 S   sleep 100
ganesh@DESKTOP-79N3772:~$ kill 12400
-bash: kill: (12400) - No such process
ganesh@DESKTOP-79N3772:~$
```

```
ganesh@DESKTOP-79N3772:~$ kill 12400
-bash: kill: (12400) - No such process
ganesh@DESKTOP-79N3772:~$ ls job_table.txt
job_table.txt
ganesh@DESKTOP-79N3772:~$
```

## TASK 2 — Final Observation

**Observation:** A job table was created to store process information, process groups and states were monitored using `ps`, process status was updated using `kill`, and completed job records were validated and removed.